PaperPass[免费版]AIGC检测报告

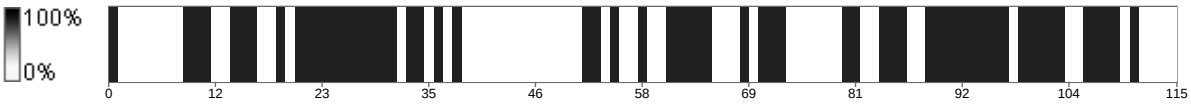
简明打印版

AIGC总体疑似度(高+中+轻)：39.97%
AIGC总体疑似度(高+中+轻)：25.77%（加权计算）

高度疑似AIGC占全文比：	0.54%
中度疑似AIGC占全文比：	29.7%
轻度疑似AIGC占全文比：	9.73%
不予检测文字占比：	22.47%

检测版本：免费版(仅检测中文)
报告编号：6XJB69F991652CC57
论文题目：thesis
论文作者：佚名
论文字数：33326
段落个数：200
句子个数：941
片段个数：115
提交时间：2026-5-5 14:42:45
查询真伪：https://www.paperpass.com/check

疑似度分布图：



正文中片段的不同颜色表示不同的疑似度范围

- 红色：AIGC生成疑似度在70%以上（高度疑似）
- 橙色：AIGC生成疑似度在60%~70%（中度疑似）
- 紫色：AIGC生成疑似度在50%~60%（轻度疑似）
- 黑色：AIGC生成疑似度在50%以下
- 灰色：过短片段、标题和英文等不予检测的文字

注：AIGC检测可有效识别文本是否部分或全部由AI模型生成，检测结果与论文质量无关、仅表示论文中内容片段存在AI生成可能性的概率。

加权计算公式：（片段1疑似度 + 片段2疑似度 + ... + 片段n疑似度）/ n
片段疑似度范围0.0~1.0，合格片段按照0计算，不计算不予检测文字

本科毕业设计（论文）

个人健康管理智能体的设计与实现

Design and Implementation of a Personal Health Management Agent

院 系： 计算机科学与技术学院

专业班级： 计算机科学与技术 2105 班

姓 名： 袁立沛

学 号： U202217212

指导教师： 邱德红

2026 年 5 月 8 日

学位论文原创性声明

本科毕业设计（论文）

AI 62%

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

学生签名：

日 期： 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保障、使用学位论文的规定，同意学校保留并向有关学位论文管理部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权省优秀学士学位论文评选将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于

1、保密 □，在_____年解密后适用本授权书。

2、不保密 □。

（请在以上相应方框内打”√”）

学生签名：

指导教师签名：

日 期： 年 月 日

摘　　要

中国有超过 2.45 亿高血压患者，但知晓率仅 51.6%，治疗率 45.8%，控制率更是只有 16.8%——这意味着超过八成患者最终没能把血压管住。现在不少家庭都备了电子血压计，可测完之后呢？多数人看眼数字就过去了，随便记在纸上，没几天就忘了搁哪儿了。即便有认真记录的习惯，对着一串高低起伏的数值，也很难说清楚这些数字到底在告诉自己什么。至于上网搜健康建议，搜出来的东西鱼龙混杂，普通人压根分不清真假。

本文做了一个叫 HealthAgent 的个人健康管理工具。它想解决的事情不复杂：拍照记下血压、打开首页看趋势、遇到困惑问 AI。后端用 Python 的 FastAPI 框架加 SQLite 数据库，前端是 React 配合 Ant Design 和 ECharts 搭起来的界面。

OCR 模块是全文最有意思的部分。我们试了好几种方案之后发现，通用 OCR 在血压计屏幕上的识别效果很差——七段数码管那种字体跟通用 OCR 训练集里的印刷体差异过大。后来我们走了一条双通路的路线：常规情况交给百度云的数字专用 OCR，速度很快（通常不到一秒）；遇上识别不全的字段，再由通义千问的视觉大模型进行补充识别。在 4 张真实的血压计屏幕照片上测试，这个组合方案做到了 75.0% 的字段准确率，跟纯PaddleOCR 的 0% 和纯百度云的 8.3% 比起来，改善明显。智能问诊这一块，我们基于 12 篇医学相关资料建了向量知识库，Prompt 中嵌入用户近 14 天血压概况，同时设置了六条安全限制——不推荐具体药量、不下诊断结论、事实必须标注引用来源等。自动评测脚本验证下来，引用标注、免责声明和剂量安全三项指标都达到了 100%。

测试方面也做了一些工作。后端 86 个测试用例全部通过，代码覆盖率 82%；前端 15 个用例也全部通过，TypeScript 严格模式下无类型错误。

关键词：健康管理智能体；OCR 图像识别；大语言模型；检索增强生成；时序预测

Abstract

China has over 245 million adults with hypertension, yet the numbers drop sharply at each step of care: 51.6% are aware of their condition, 45.8% get treated, and a me 16.8% achieve control. Home blood pressure monitors are commonplace, but most users lose track after taking a reading —writing numbers on scraps of paper that get lost, staring at a list of values without knowing what the trend means, or searching online only to wade through unreliable advice.

We built HealthAgent to close this gap. The core workflow is straightforward: photograph the monitor screen, check a dashboard that shows your trends, and consult an AI that grounds its answers in medical references. Python FastAPI with SQLite runs the backend; the frontend combines React 18, TypeScript, Ant Design 5, and ECharts.

The OCR component proved to be the most instructive part of this project. General-purpose OCR models perform surprisingly poorly on blood pressure monitor screens —those seven-segment digits bear little resemblance to the printed text found in standard training corpora. After trying several approaches, we settled on a dual-pathway design: Baidu Cloud’ s number-specialized OCR serves as the fast primary path (under one second per call), while Tongyi Qianwen’ s vision-language model acts as a fallback when any required field comes up missing. Across four real monitor photos with twelve annotated fields, this combination reached 75.0% accuracy, a substantial improvement over the 0% from PaddleOCR alone or the 8.3% from Baidu’ s general OCR. For the health consultation module, twelve curated medical documents were embedded into a vector store. Each query prompt bundles the user’ s 14-day blood pressure summary alongside retrieved knowledge chunks, with six safety constraints baked into the system instructions —no dosage specifics, no diagnostic conclusions, mandatory source citations, and a required disclaimer on every response. An automated evaluation harness confirmed these three safety metrics at 100% across ten test questions.

We built a layered test suite for verification: 86 backend tests pass with 82% li coverage, and 15 frontend tests pass with zero TypeScript strict-mode errors.

Keywords: Health Management Agent; OCR Image Recognition; Large Language Model; Retrieval-Augmented Generation; Time Series Prediction

目　　录

目录

学位论文原创性声明 **1**

学位论文版权使用授权书 **2**

摘要 **3**

Abstract **4**

目录 **5**

1 绪论 **7**
1.1 研究背景与意义 7
1.2 国内外研究现状 7
1.2.1 医疗设备屏幕 OCR 7
1.2.2 健康数据趋势分析 8
1.2.3 大模型在医疗领域的应用 9
1.3 本文的主要工作 11
1.4 论文组织结构 11

2 相关技术 **12**
2.1 光学字符识别技术 12
2.2 大语言模型与 RAG 12
2.3 时序数据分析方法 13
2.4 系统实现技术栈 13

3 需求分析与系统设计 **15**
3.1 功能需求 15
3.2 目标用户与使用场景 15
3.3 非功能需求 16
3.4 系统架构 16
3.5 数据库设计 17
3.6 安全设计 18

4 关键模块实现 **19**
4.1 双通路 OCR 血压计图像识别 19
4.1.1 整体流水线 19
4.1.2 方案对比实验 20

4.1.3 人机协同录入界面 21
4.2 健康数据管理与趋势预测 21
4.2.1 数据管理与用户隔离 21
4.2.2 统计聚合 21
4.2.3 自适应窗口趋势判定 21
4.2.4 前端看板 22
4.3 RAG 大模型健康问诊 22
4.3.1 知识库构建 22
4.3.2 语义检索 23
4.3.3 安全 Prompt 设计 23
4.3.4 SSE 流式问答 23
4.3.5 自动评测 24

5 系统测试与评测 25
5.1 测试策略 25
5.2 后端测试 25
5.3 前端测试与类型检查 25
5.4 全链路端到端测试 25
5.5 RAG 问诊评测 26
5.6 性能基准 27
5.7 安全测试 27
5.8 验收结论 28

6 结论与展望 29
6.1 工作总结 29
6.2 存在的不足 29
6.3 未来展望 30

参考文献 31

附录 33

致谢 35

1 绪论

1.1 研究背景与意义

过去四十年，中国人的疾病谱变了个样。传染病退居次要位置，取而代之的是高血压、糖尿病这类需要长期管理的慢性病。按照《中国心血管健康与疾病报告 2023》的数据，18 岁以上成年人里高血压患病率约为 27.9%——四个成年人里就有一个，算下来全国患者超过 2.45 亿人，而且越来越多年轻人也开始中招。但真正让人担心的不是患病率有多高，而是控制情况有多差：知道自己有高血压的人只占 51.6%，在治疗的占 45.8%，最终能把血压管住的只有 16.8%。而大量证据已经表明，只要把血压管好，心脑血管出大事的风险就会大幅降低——收缩压每降 10 mmHg，主要心血管事件的风险大约降 20%，中风风险降 27%。

电子血压计现在很多家庭都有了，但" 测完之后怎么办" 这个问题基本没人解决。大多数人的流程是这样的：挽起袖子测一下 → 瞄一眼屏幕上的数字 → 心里大概记一下或者找张纸写下来 → 过几天就全忘了。对老年人来说就更麻烦了——眼睛看不太清那小屏幕，手也没那么灵活，每次测完要拿笔抄下三个数字，本身就是件挺吃力的事。有些高端血压计可以蓝牙连手机 App，但各家的数据不互通，换一台设备之前的历史就断了。还有个大问题是，这些 App 顶多给你画条血压曲线，但从来不告诉你这条曲线到底想说什么——某次高了一点要不要紧张？最近少吃了盐是不是真的有效果？用户只能自己猜。

AI 56%

想找专业的人问问也难。网上一搜高血压，出来的内容什么都有，大部分是卖保健品的营销号，普通人根本分不清真假。去医院问医生当然最靠谱，但挂个号排半天队就为了问几个问题，时间和精力成本都不低，在农村和偏远地区就更不用说了。

AI 68%

最近两年，大模型的能力有了显著提升。OCR 拍照识别技术已经相当成熟，拍张照就能提取出数字；RAG 技术可以将大模型的输出约束在可信的医学知识范围内，有效降低幻觉风险；流式输出则让对话体验更接近与人交流。能否将这些技术整合起来，做一个操作简单到老年人也能使用、但内核足够专业的家用工具？这正是 HealthAgent 的出发点。

AI 70%

HealthAgent 的核心功能可以概括为三项：拍照记录血压、看图理解趋势、对话获取建议。将这三个环节串联起来，便构成了从数据采集到理解分析再到行动决策的完整闭环。

1.2 国内外研究现状

1.2.1 医疗设备屏幕 OCR

OCR 这门技术其实历史很长了，从 20 世纪 60 年代开始就有人在研究。早期的做法主要是模板匹配，靠人工设计几何特征，对字体和排版要求很死。90 年代以后，统计机器学习让 OCR 的适应性上了个大台阶——Tesseract 就是个典型例子，从 1985 年 HP 实验室开始搞，中间改了三次架构，2006 年开源后成了用得最广的通用

OCR 引擎。再到深度学习时代，CNN 加 CTC 的方案大幅提升了场景文本识别水平，CRNN+CTC 这套组合一度是行业标配。2020 年以后，百度的 PaddleOCR 系列靠轻量化的设计在移动端部署上表现不错，v4 版本在通用中英文场景的识别率能做到 95% 以上。

但把通用 OCR 拿来识别电子血压计屏幕上的数字，事情就没那么简单了。我们实际操作下来发现有几个很头疼的问题。

第一个是显示介质的特殊性。家用血压计的屏幕大多是七段数码管或者低分辨率 LCD 屏，数字的形状和通用 OCR 训练集里那些规整的印刷体差了太多——数码管的”1” 很容易被看成”l”，”8” 被当成”0” 或”B”；LCD 屏上的”6” 和”5” 笔画出点断裂，识别结果就全乱了。我们用PaddleOCR PP-OCRv4 在 4 张血压计屏幕照片上做了测试，12 个字段无一识别正确，准确率为 0%。进一步分析发现，这是因为模型的训练数据中几乎不包含七段数码管这类字体样本。

第二个是拍摄条件完全没法控制。用户拍照的时候什么角度、什么光线都有，反光、阴影、手抖模糊全来了——这和文档扫描那种在受控环境下拍照完全不是一回事。

第三个是” 谁是谁” 的问题——屏幕上除了收缩压、舒张压、心率，还有时间、电池图标、记忆编号等各种东西。机器要能分辨哪些数字是真正的血压读数，哪些只是杂七杂八的辅助信息。

AI 68%

这三个问题单独看都不算太难，但叠加在一起就构成了一个典型的” 数据少、场景多变” 的工程挑战。

目前市面上的方案大致分为两类。传统OCR 引擎（Tesseract、PaddleOCR）的优势在于速度快、成本低，但面对非标准字体时表现明显下降——Tesseract 的 LSTM 识别层虽然经过大量合成数据训练，具备一定的字体适应能力，但七段数码管与其训练数据的分布差异过大，识别效果仍然不理想。另一派是多模态视觉大模型（GPT-4V、通义千问 VL、Gemini Pro Vision），它们不只是” 看” 数字长什么样，还会结合屏幕布局、字段之间的关系来综合判断，鲁棒性强了不少。但麻烦在于每次调用要花 2–3 秒，API 费用也比专用 OCR 贵几十倍——血压这种东西几乎天天要测，按这个频率用 VLM 成本扛不住。

AI 68%

我们最终选择了中间路线：常规情况由百度云数字专用 OCR 快速处理，识别不全时再由 VLM 补充缺失字段。这样在速度和成本之间取得了平衡，同时 VLM 为困难样本提供了额外的识别能力。类似这种” 主力加替补” 的协作模式在工业界较为常见，但把它搬到家用医疗设备屏幕识别这个细分场景，并且给出定量的对比实验数据，算是本文的一个小贡献。

AI 68%

1.2.2 健康数据趋势分析

面向个人的健康数据管理工具现在选择已经不少了，国内外的产品定位各有侧重。

国际上，Apple Health 通过 HealthKit 把 iPhone、Apple Watch 和第三方设备的数据汇到一起，心率、血压、血氧、睡眠等几十项指标都能看。但它的重点是把数

据聚起来，不是帮你分析——它能告诉你”上个月平均收缩压 135 mmHg”，但不会接着解释这意味着什么、你该做什么。Google Fit 在 2022 年改版后引入了 WHO 和 AHA 的运动建议作为活动目标参考，在慢性病管理上也没有深入太多。硬件厂家这边，Omron Connect 配合欧姆龙蓝牙血压计可以自动同步数据、画基础图表，但离开欧姆龙的设备就用不了；Withings Health Mate 走的是多品类路线（血压计、体重秤、手表都做），生活建议方面做得细致一些，但中文适配和国内本地化都不太到位。

国内的话，华为运动健康和小米健康各自有大量穿戴设备用户。华为在血压这块跟多家医疗机构合作做高血压管理研究，部分型号手表已经支持腕式血压测量和血压健康研究计划——这个”被动测量”的思路挺巧妙，用户日常戴着手表数据就自动沉淀下来了，不需要专门用上臂式血压计去测。小米健康则通过跟小爱同学整合，让老年用户直接用语音就能操作，门槛低了不少。但总的来说，这些平台的重心还是放在数据采集和简单统计上，数据解读层面大多也就是”本月血压偏高次数：X 次”，缺少对个人趋势更深度的分析，更不会主动给出什么健康建议。

在趋势预测方法这块，学术界主流的探索方向有三个。Prophet 是 Facebook 在 2017 年开源的，它把时间序列拆成趋势项、周期项和节假日效应三部分来做预测，对缺失值和异常值不太敏感——这对家庭场景下采样不规律的问题确实有一定缓解。但 Prophet 有个前提：至少需要两个完整周期的数据（一般要 90 天以上）。新用户或者不怎么测的用户根本达不到这个门槛。LSTM 靠门控机制能抓比较长的时间依赖，在 ICU 患者生命体征预测这种医疗时序任务上效果很好；问题是家庭用户的自测数据太少了（通常就几十条到几百条），在一个用户身上单独训 LSTM 铁定过拟合，而跨用户训又面临隐私和用户间差异的问题。ARIMA 很经典，理论基础扎实、结果好解释，但它要求序列平稳——血压数据一般都不平稳，要先做差分处理，而差分的阶数和滞后项怎么选对结果影响很大，这些前处理步骤对没有统计背景的人来说门槛不低。

AI 54%

这几种方法在家庭场景下面临一个共同的瓶颈——模型需要大量数据才能输出可靠的预测，但真实的家庭血压数据恰恰是稀疏且不规律的。许多用户并非每日测量，新用户更是只有寥寥几条记录。2022 年 JMIR 发表的一篇系统综述回顾了 39 项面向消费者的血压管理 App 研究，结论是”绝大多数应用仅提供数据可视化，不到 15% 的应用具备任何形式的预测或趋势分析功能”。这一结论与我们在项目启动前所做的竞品调研结果一致。基于此，我们的设计思路是：当数据不足以支撑可靠预测时，诚实地告知用户”需要继续测量”，同时将目前能给出的统计信息完整呈现；等数据积累充分后，再提供有统计意义的趋势判断。后文将要介绍的趋势判定算法，正是按照这一”先确保不误导、再争取提供帮助”的原则来设计的。

1.2.3 大模型在医疗领域的应用

大语言模型在医疗健康领域是近两年最活跃的研究方向之一。2022 年底 ChatGPT 让公众第一次直观感受到大模型回答医学问题的能力，但学术界的探索在此之前就已展开。2018 年，Google 在 BERT 基础上通过生物医学文献继续预训练得到了 BioBERT 和 PubMedBERT，在命名实体识别、关系抽取等 NLP 任务上刷新了

当时的记录。不过这两个模型止步于分类和抽取，无法生成自由文本，不能直接用于问诊。2020 年 GPT-3 的出现改变了局面——它在零样本和少样本条件下的泛化表现令人意外，模型内部积淀的医学常识量也相当可观。到 2023 年 7 月，Google 和 DeepMind 发布了专门面向医疗场景的 Med-PaLM 2，在 USMLE 类问题上准确率达到 86.5%，已接近执业医师水平。相比前代，它在回答一致性、事实准确性和安全性方面都有显著改进，尽管距离真正的临床应用仍有距离。中文领域同样进展迅速：HuatuoGPT 用中文医学对话数据做了微调，在知识问答和医患对话模拟上表现良好；BenTsao（本草）则专攻中医药知识和中西医结合问答，填补了通用模型在中医药领域的空白。

然而，多项研究也不约而同地指出了三个无法回避的问题。

最棘手的是幻觉。大模型有一种令人不安的能力——用极其笃定的语气输出完全错误的信息。日常闲聊中说错话也许无关紧要，但在医疗场景下，一个错误的药物剂量建议可能造成严重后果。Stanford HAI 在 2024 年发布了一份医疗 AI 可信度评估报告，指出即便最好的模型，在医学事实上仍有 3–5% 的错误率，更麻烦的是，这些错误偏偏集中出现在模型表现得”最有把握”的回答中——用户仅靠自己的判断根本无法识别。

其次是知识时效性问题。模型的知识在训练完成那一刻就凝固了，此后的新指南、新证据一概不知。一个现实例子是，2024 年发布的《中国高血压防治指南》修订版在血压分级标准和用药推荐上都做了不小的调整，但训练语料截止于 2022 或 2023 年的模型对此一无所知。

第三个问题是缺乏个性化能力。通用模型的经验源于”平均人”——它倾向于给出放之四海而皆准的建议，却很难结合用户具体的血压历史、年龄、性别、合并症等信息做差异化判断。一个 25 岁的健康人和一个 70 岁伴有糖尿病的老人，两人的”安全血压范围”可能相差 10–15 mmHg，但通用模型很少能在对话中敏锐地做出这种区分。

Lewis 等人在 2020 年 NeurIPS 上提出的检索增强生成（RAG）为上述三个问题提供了一条工程上可行且成本可控的解决路径。其核心思路并不复杂：每次回答问题之前，先从知识库中检索与用户问题最相关的文档片段，然后将这些片段连同系统指令和用户数据一起组装成 Prompt，再送入大模型。这相当于在每次对话前给模型做一次”临时的知识补充”——它不必在训练阶段记住所有信息，推理时”读一遍”检索到的材料即可。RAG 抑制幻觉的机制是让模型输出受限于可验证的外部知识边界；解决知识时效性的方式是知识库可以随时更新而无需重新训练模型；个性化则通过将用户自身数据嵌入 Prompt 来实现。

相比微调模型，RAG 在工程层面有几个明显优势：修改知识只需更新文档，不用重训模型；每一条事实性陈述都能回溯到知识库中的具体来源——这对医疗场景尤为关键；部署成本也远低于维护一套定制微调模型的 GPU 集群。2023 年以来，LangChain 和 LlamaIndex 等 RAG 编排框架快速成熟，已在企业知识问答、法律文书检索、医疗咨询辅助等多个领域落地。但这些框架的抽象层级较高，随之而来的问题是 Prompt 结构无法完全受控，Token 消耗也不够透明。基于这些考虑，我们最终

选择了自行实现 RAG 编排逻辑——多写了一些代码，但换来了对安全约束的精确控制。

AI 70%

1.3 本文的主要工作

本文的工作围绕以下几条主线展开。

首先是 OCR 模块。我们搭了一套双通路的血压计屏幕识别方案：百度云数字OCR 作为主通路处理常规情况，通义千问视觉大模型在识别不完整时进行补充识别。AI 66%
两个通路在字段级别做合并——VLM 只填补主通路未检出的字段，不会覆盖已正确识别的部分。另一个关键设计是，识别结果不直接入库，而是先回填到前端表单供用户逐字段核对和修改，确认无误后再写入。在健康场景下，一个被认错的数字比一个没认出来的数字风险更大，让用户做最终把关是更稳妥的做法。

AI 69%
第二块是面向安全的个性化 RAG 问诊系统。我们用 12 篇医学知识文档构建了向量知识库（通过 sqlite-vec 扩展实现向量存储和检索），Prompt 中嵌入两块动态信息——用户最近 14 天的血压概况和检索到的知识库片段。系统指令中内嵌了六条安全红线：禁止推荐具体药物剂量、禁止下诊断结论、事实必须标注引用来源、每条回复必须附带免责声明等。此外，我们写了一套自动评测脚本，每次修改Prompt 后可以快速重新验证安全指标，不用依赖人工逐条审查。

AI 55%
第三块是端到端的数据链路——从采集、趋势分析到可视化看板。这包括用户级数据隔离、按数据量自适应调整窗口的趋势判定算法，以及基于 ECharts 的双 Y 轴时序看板。

最后是分层的测试体系（单元 → 集成 → 端到端）。后端86 个测试用例全部通过，代码覆盖率 82%；前端 15 个用例也全部通过，TypeScript 严格模式无类型错误。

AI 51%

1.4 论文组织结构

本文后续章节的安排是：第二章说用到哪些关键技术以及为什么选它们；第三章做需求分析和系统架构设计；第四章讲三个核心模块具体怎么实现的；第五章是测试策略和评测结果；第六章总结一下做了什么、还有哪些地方可以接着做。

2 相关技术

这一章介绍 HealthAgent 用到的几项核心技术。和常规综述不太一样，我不会只罗列技术概念，而是同时说明在这个系统里为什么选了它、没选别的——每个选择背后都有具体的场景约束。

AI 61%

2.1 光学字符识别技术

OCR 的完整流程一般分四步：图像预处理、文本检测、字符识别、后处理。我们这个系统里，每一步都针对血压计屏幕这个特殊场景做了调整。

预处理这块，最大的麻烦是用户拍照的条件完全没法控制。我们的处理顺序是这样的：输入图像（支持 JPG/PNG/WebP 三种格式）先转成 RGB 数组，然后按长边等比缩到最长不超过 1920 像素（留够细节又不至于太大），接着转灰度图降维度，最后用 CLAHE（限制对比度自适应直方图均衡化）提一下局部对比度。CLAHE 和普通的全局直方图均衡化不一样的地方在于，它把图像切成 8×8 个小块，每个小块里单独做均衡化，再通过双线性插值平滑拼回去，同时设了个对比度上限（clip limit=2.0）防止噪声被过度放大。实测下来，加 CLAHE 对缓解屏幕反光和光线不匀的问题帮助不小。

文本检测和识别这块，我们主要用的是百度云的数字 OCR 接口（numbers 接口）。这个接口是专门为仪表、计数器、液晶屏之类数字识别场景做的优化，返回的结果里每个数字都带归一化的边界框和置信度。之所以选择云端 API 而非本地部署模型，是因为我们测试发现，PaddleOCR 在本地运行时对七段数码管的识别效果很不理想。要使其在血压计屏幕场景下达到可用水平，需要专门收集大量标注样本进行微调——这一工作量对于本科毕设而言不太现实。

AI 51%

后处理是整个过程里看似简单却最为关键的步骤。百度云返回的是一组未经排序的识别结果，需要从中筛选出真正需要的三个数值（收缩压、舒张压、心率），同时排除时间戳、电池图标等干扰项。我们定了四条规则：先按边界框高度把明显太小的字筛掉（辅助信息一般都比血压读数小很多）；再按 Y 坐标排序，利用屏幕的固定布局分配字段（上 → 收缩压、中 → 舒张压、下 → 心率）；然后用生理合理范围做数值校验；最后用”收缩压必须严格大于舒张压”这个医学常识做一致性检查。经过这四步过滤，大部分噪声信息能够被有效排除。

AI 69%

2.2 大语言模型与 RAG

大语言模型底层是 Transformer 解码器架构，通过在海量文本上做自回归预训练来学习语言理解和生成能力。生成过程是逐Token 自回归的——每一步根据上文计算下一个 Token 的概率分布，再按照一定策略采样。我们选的是通义千问的 qwen-plus 模型，理由比较务实：接口兼容OpenAI 格式，方便以后切换模型；中文医学术语覆盖得好；原生支持流式输出；毕设期间有免费额度可用。

不过把通用大模型直接用在医疗场景下有个致命问题——幻觉。模型会用斩钉截铁的语气说出完全不正确的内容。日常闲聊说错一两句不算什么，但医疗场景下，一条错误的药物剂量建议真的可能带来严重后果。

RAG（检索增强生成）针对这个问题提供了一条可行的出路。它的思路是：每次对话时，先让用户问题去知识库里做一次检索，把最相关的文档片段找出来，然后把这些片段和系统指令、用户数据拼到一起组成完整的 Prompt，再发给大模型。这样模型在推理时”读一下”相关资料就能回答问题，不用在训练阶段记住所有知识。RAG 的核心优势在于，每条回答都能追溯到知识库里的具体文档来源，幻觉风险因此大幅降低；而且知识库更新后不需要重新训练模型。

我们没有用 LangChain 这类现成的编排框架，而是自己写了 RAG 的编排逻辑。主要考虑是：Prompt 模板的每一行都能完全掌控，六条安全红线可以逐条审计验证；Token 消耗一目了然，不会被框架悄悄吃掉；同时也少引入了一堆第三方依赖。代价是多写了些代码，但换来了安全层面的确定性，这笔账是划算的。

2.3 时序数据分析方法

血压趋势判定上，我们走的是轻量路线——滑动平均配合线性回归，没上复杂的时序模型。这个取舍的出发点很现实：家庭场景下的血压数据量通常非常有限。新用户可能只有寥寥几条记录，即便是老用户也不一定天天测，采样间隔完全随缘。

滑动平均（SMA）大概是所有平滑方法里最朴素的一个——取一个固定大小的窗口在序列上滑过去，对窗口里的值求算术平均。它的好处一目了然：计算量小，结果好理解。在前面这个基础上用线性回归拟合一条趋势线，斜率的正负和绝对值大小就能大致反映血压的整体走向：是往上走、往下走，还是原地踏步。

Prophet 和 LSTM 这类更复杂的模型在学术界当然各有各的用武之地，但它们都假设你手上有成百上千个数据点——这个前提在家庭自测场景下根本不成立。Prophet 需要至少 90 天的连续序列才能可靠地估计周期参数；LSTM 在几十条数据上做训练，得到的结果往往还不如一个简单滑动平均来得稳定。我们在整个系统中坚持一条原则：数据够了就给趋势判断，数据不够就坦诚地告知用户”还需要继续测量”，绝不强行输出一个统计上站不住脚的预测。

2.4 系统实现技术栈

后端用的 Python 3.11 加 FastAPI。为什么选 FastAPI？我们在 Flask、Django 和 FastAPI 之间比较了一下。Flask 很轻很灵活，但它的函数视图在项目大了之后要靠 Flask-RESTful 这些扩展才能自动校验请求参数，类型安全基本靠自觉。Django 什么都有——ORM、管理后台、认证系统全自带，但对这个项目来说反而太重了，我们不需要模板引擎（前端是独立 React 应用），也不需要管理后台。FastAPI 刚好夹在中间：靠 Python 类型注解和 Pydantic v2 自动生成请求校验、序列化和 API 文档，同时原生支持 `async/await`——这对需要同时等百度云 OCR 和 DashScope 大模型返回的场景确实有用。

数据库选了 SQLite，通过 SQLAlchemy 2.0 ORM 操作。很多人觉得 SQLite 是”玩具数据库”，但在单机部署场景下它其实是个很务实的选择。MySQL 和 PostgreSQL 需要独立的服务进程——安装、配置、权限管理、日常运维每样都有门槛，这跟” 面向普通家庭、部署简单” 的目标是对着干的。SQLite 整个数据库就是一个文件，不需要任何服务进程，备份只要复制文件；而且支持 WAL 模式，读写并发性能在家庭场景下绰绰有余。更关键的是 sqlite-vec 向量扩展——能把向量数据和关系数据放在同一个数据库文件里管理，省去了 Elasticsearch 或 Pinecone 那种外部向量数据库的部署和同步麻烦。向量存储用的是 sqlite-vec 的 vec0 虚拟表，支持 L2 距离和余弦距离，1024 维的文本嵌入向量建了索引后单次 Top-5 查询一般在几十毫秒内完成。

前端选择了 React 18 配合 TypeScript。客观而言，Vue 3 的 Composition API 对新手更加友好，模板语法也更贴近传统 Web 开发；Angular 则提供了完整的企业级方案，但框架本身的体量较大。最终选择 React 主要基于两点：Ant Design 5 和 ECharts 在 React 生态中的封装最为成熟；TypeScript 的类型推导在 React+hooks 模式下支持得最好。组件库用了 Ant Design 5——它的 ConfigProvider 主题定制机制可以在全局统一设 Design Token（颜色、圆角、字号、间距等），以后想调整整体视觉风格只需要改一个地方，20 多种预置组件覆盖了系统需要的各种界面元素。数据可视化用的是 ECharts 的 echarts-for-react 封装，血压看板的双 Y 轴折线图（左轴 mmHg、右轴 bpm）加三色异常值标注就是基于它做的。跟 D3.js 需要手写几十行 SVG 操作代码比起来，ECharts 的声明式配置让可视化的开发轻了不少。

安全认证用了 JWT 双 Token：Access Token 15 分钟有效，用于日常接口鉴权；Refresh Token 7 天有效，用于无感续期。做成两个 Token 而不是一个长有效期的，主要是想在安全和体验之间取个平衡——如果只用一个长期 Token，泄露了整个账户就暴露了；短周期 Access Token 能把泄露的影响窗口控制在 15 分钟以内。密码用 bcrypt 哈希存（cost factor=12），这个参数在安全和登录延迟之间折了个中——cost factor 为 12 时单次哈希大约 250ms，暴力破解的成本则被推到了不现实的程度。

3 需求分析与系统设计

3.1 功能需求

按照毕设任务书的要求，系统要覆盖图像识别、数据管理、智能问诊三块能力。经过梳理，最终定了七个功能需求：

用户注册与认证。支持用户名和邮箱注册，密码登录后返回 JWT 双 Token。用户可以维护个人资料（昵称、性别、年龄），改密码要验证当前密码。Token 过期了通过 Refresh Token 静默续期，用户不用反复登录。

血压计图像 OCR 识别。用户上传血压计屏幕照片（支持 JPG/PNG/WebP，上限 10MB），系统自动识别收缩压、舒张压、心率三项。识别结果用彩色Tag 标完整度（绿色 = 三项全了、黄色 = 缺一两项、红色 = 一个没认出来），数值自动填回前端表单让用户逐字段确认或修改。

手动与 OCR 辅助录入。除了拍照录入，也提供独立的手动录入入口。OCR 的结果先填回表单、用户确认后再写入数据库——这条” 机器先认、人来把关” 的闭环是系统设计的一个核心理念。

健康数据看板。首页看板聚合展示近期血压均值、最高值、最低值和超标记录数，配一张 ECharts 双 Y 轴时序折线图（收缩压/舒张压在左 mmHg 轴、心率在右 bpm 轴），支持7 天/30 天/90 天窗口切换。

趋势判定。基于自适应窗口的滑动平均算法给出短期趋势（上升/下降/平稳）。数据不够 3 天的时候直接标” 数据不足”，不强行做判断。

智能健康问诊。用户在对话界面输入健康问题，系统从知识库里搜相关资料，结合用户近 14 天血压数据，通过大模型生成个性化回复。回复以 SSE 流式逐字输出（打字机效果），每条回复标出引用的知识库来源，支持随时取消。

会话管理。可以创建多个独立对话、切换会话、看历史消息、删会话，对话记录持久化保存。

3.2 目标用户与使用场景

在细化需求之前，先把系统的目标用户想清楚，很多设计选择后面都能说得通。HealthAgent 主要面向三类人：

居家慢病管理的中老年人。这是最核心的用户群。典型画像就是 55–75 岁的原发性高血压患者，家里有电子血压计，每周测 2–5 次，但没有长期记录的习惯。对手机玩得不太熟，最想要的就是” 拍个照就能记下来、点个键就能看到趋势”。对这类用户来说，最核心的需求其实是降低录入的阻力——手动输三个数字（收缩压、舒张压、心率）对年轻人就是几秒钟的事，但对眼睛花、手指不灵活的老人来说，每次测量后的录入都是一次小小的受挫。

关心父母健康的年轻人。通常 25–40 岁，跟父母分居两地，父母有高血压或其他慢性病。想了解父母的血压变化趋势，出问题了能及时知道，但不想给父母添太多操

作负担。这类人的价值在于：他们往往是 App 的实际安装者和初始配置人，会帮父母完成第一次设置。同时他们对”数据共享”“家庭组”“异常提醒”这些更高级的功能也有明确需求。

高血压初诊或临界人群。典型画像就是体检发现血压偏高（比如 130–140/85–90 mmHg）但还没确诊的中青年人。医生建议”在家先监测一段时间看看”。这类人对血压知识了解不多，有各种具体但零碎的问题——”早上测还是晚上测？”“测之前能喝咖啡吗？”“140 算不算高？”——通常也不会为这些问题专门去挂个号。他们最想要的是：打开首页能看到直观的趋势变化，在问诊界面随时能得到靠谱又不吓人的解答。

基于这三类人，交互上做了几个针对性的选择：OCR 拍照作为默认录入方式（降低老人的手动输入成本）；首页看板用彩色卡片和高亮异常值突出关键信息（少看字，多看图）；问诊界面用对话式交互降低学习门槛（不需要懂菜单结构，会打字就行）。

3.3 非功能需求

除了功能上的要求，考虑到系统面向的是医疗健康场景，还有几条硬性约束必须满足。

性能指标。OCR 从上传到返回结果不超过 3 秒；问诊首字延迟不超过 3 秒；看板页面加载 1000 条记录时首屏渲染不超过 1 秒；数据库查询不超过 100ms。

安全与数据隔离。所有用户数据严格按 user_id 做隔离，任何跨用户访问统一收到 HTTP 404 响应——这里刻意不用 403，是为了防止有人通过状态码差异来探测”某条记录到底存不存在、只是不属于我”。密码经 bcrypt 哈希后存储，绝不落盘明文。所有业务接口强制通过 JWT 依赖注入进行鉴权。

医疗合规约束。AI 生成的全部内容必须附加免责声明（”以上信息仅供参考，不构成医疗诊断”）。模型被明确禁止输出具体药物的用法用量，也不允许使用确定性诊断语气（比如”你患有 XX 病”）。所有事实性陈述必须标注知识库引用来源编号。

代码质量标准。后端单元测试覆盖率不低于 70%；前端 TypeScript 严格模式零类型错误；全链路端到端集成测试覆盖核心用户路径。

3.4 系统架构

HealthAgent 用的是很经典的前后端分离三层架构。

数据层负责持久化存储。关系型数据（用户、血压记录、会话消息）用 SQLite 通过 SQLAlchemy ORM 操作，向量数据（知识库文档的嵌入向量）用 sqlite-vec 扩展的 vec0 虚拟表存储，两者共存于同一个 SQLite 文件——不需要单独部署什么中间件，备份也很简单。文件存储（用户上传的血压计照片）直接用本地文件系统，以后如果需要可以换成对象存储。

服务层是业务核心。各个模块职责是分开的：user_service 处理认证和资料管理；ocr_service 编排从图像预处理到 VLM 兜底判断的完整 OCR 流水线；bp_record_service 实现血压记录的增删改查、统计聚合和趋势预测；chat_service 编排 RAG 全流程（检

索 →Prompt 组装 →SSE 流式生成）；chat_store 负责会话和消息的持久化。模块之间通过依赖注入传递数据库会话和配置，互相之间耦合不紧。

接口层基于 FastAPI 对外暴露 RESTful API，统一以/api/v1 为版本前缀，一共 18 个端点。接口层主要做三件事：用Pydantic Schema 校验请求参数、通过 Depends 依赖注入做 JWT 鉴权、把服务层的异常统一转成 HTTP 错误响应。API 文档由 Swagger UI 和 ReDoc 自动生成。

前端是 React 单页应用，React Router 6 管路由、Zustand 管全局状态、Axios 做 HTTP 通信。组件按功能分成页面级（登录、看板、录入、对话等）和通用组件（鉴权守卫、布局框架、错误边界等）。

系统整体架构如图1所示。

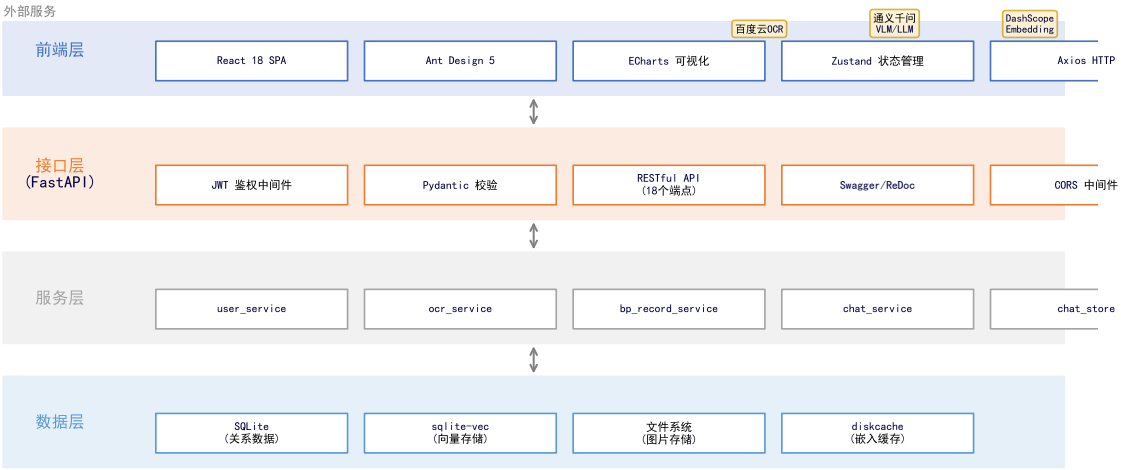


图3-2 HealthAgent系统架构

图 1　HealthAgent 系统架构

3.5　数据库设计

核心数据库表围绕四组业务对象来组织。

用户与认证。users 表存放用户基本信息以及 bcrypt 哈希处理后的密码。birth_date 字段用于计算年龄，在后续做个性化血压阈值判定时会用到。

血压记录。bp_records 表是整个系统最核心的业务表。measured_at 字段上建了降序索引，这是为了优化按时间倒序分页——查询血压历史是最高频的操作。source 字段限定在 manual 和 ocr 两个取值；image_id 可关联到用户上传的原始照片，方便追溯。

会话与消息。conversations 和 chat_messages 两张表管理问诊对话的全生命周期。chat_messages 中有一个 citations_json 列，以 JSON 数组格式存储每条回答的引用详情（chunk 编号、所属文档标题、向量距离分数），前端据此渲染引用来源卡片，让用户看到每条信息的出处。

知识库。kb_documents、kb_chunks、kb_vectors 三张表构成了知识库的三层存

储体系。kb_documents 利用 content_hash 做增量去重——内容相同的文档不会重复入库，避免知识库膨胀。kb_vectors 是 sqlite-vec 扩展提供的 vec0 虚拟表，存储 1024 维的文本嵌入向量并维护索引。

AI 68%

3.6 安全设计

认证安全方面，密码用 bcrypt（cost factor=12）做哈希，JWT 密钥是随机生成的 256 位字符串，Access Token 有效期 15 分钟。授权安全方面，所有业务接口通过 get_current_user 依赖注入强制鉴权，服务层对资源归属做二次校验（比记录的 user_id 和 JWT 里的 user_id 是不是一致），防止不安全的直接对象引用（IDOR）。

AI 65%

数据安全方面，Pydantic 输入 Schema 对所有字段做严格白名单校验，密码字段永远不会出现在任何 API 序列化输出中，分页查询设了上限防止资源耗尽。

4 关键模块实现

本章展开讲三个核心模块的具体实现——这部分涉及大量工程细节，可能是整篇论文里最不”干净”但也最接近实际情况的内容。

4.1 双通路 OCR 血压计图像识别

这是数据采集的入口，也是整个项目里我最花时间折腾的部分。

4.1.1 整体流水线

OCR 模块的处理流水线一共四个串行阶段，VLM 兜底只有在主通路产出不全的时候才触发：

图像预处理。输入图像先做格式标准化（统一解码成 RGB 数组），然后长边等比缩到最长 1920 像素，接着转灰度图，最后上 CLAHE 提对比度。这套预处理不是一开始就想好的——最早我们只做了灰度化加二值化，结果发现反光重和光照不足的样本效果很差。后来试了 CLAHE，百度云 OCR 的检出改善肉眼可见。

主通路 OCR 调用。预处理后的图像用 Base64 编码传到百度云数字 OCR 接口（numbers 接口）。这个接口专门针对仪表、液晶屏之类场景做了训练，对七段数码管有一定适应能力。返回的 JSON 数组里每个元素包含识别出的文字、归一化边界框坐标和置信度。单次调用一般在 0.8 秒以内。

AI 55%

结构化字段抽取。这是整个流水线里看似简单却最为关键的环节。百度云返回的结果是杂乱排列的——血压值、时间戳、电池标志、记忆编号全部混在一起。我们设计了四步规则从中筛选出真正需要的三个数值：

第一步，按边界框高度过滤。算出所有识别结果的高度中位数，只保留高度超过中位数 70% 的。血压读数是屏幕上最大的数字，通常占屏幕高度30–40%，而时间戳（比如”08:23”和电池标志的字号明显小得多。

第二步，按 Y 坐标排序。把留下的数值按纵坐标从小到大排。家用血压计的屏幕布局很固定——收缩压在上面、舒张压在中间、心率在下面——排好序后按位置分配字段就行。

第三步，数值范围校验。每个候选值必须落在临床合理范围：收缩压 60–250 mmHg、舒张压 30–200 mmHg、心率 30–220 bpm。超范围的值直接当噪声扔掉。

AI 61%

第四步，字段一致性校验。如果抽出来的收缩压≤ 舒张压，那就扔掉舒张压——因为从生理学定义上收缩压必须严格大于舒张压。这个简单的约束能拦掉不少字段归属错误。

VLM 兜底复检。当主通路有任一必需字段未能识别、或一致性校验未通过时，触发通义千问 VLM（qwen-vl-plus）进行补充识别。调用 VLM 时使用的 Prompt 经过专门设计，要求返回 JSON 格式的三个字段，并明确指示”若看不清楚则填写 null，不要猜测”。VLM 的返回结果按字段级别覆盖主通路——仅替换主通路未能识别或校验失败的那个字段，不会影响已正确识别的部分。

4.1.2 方案对比实验

我们在 4 张真实的家用血压计屏幕照片上（含不同光照和屏幕类型），对五种方案做了端到端对比。结果如表1和图2所示。

AI 70%

表 1 OCR 方案对比实验数据

方案	字段准确率	平均耗时 (s)
PaddleOCR PP-OCRv4（CPU，通用中英）	0/ 12 (0.0%)	2.42
PaddleOCR + CLAHE/反色三变体	1/12 (8.3%)	4.32
百度云 accurate 通用 OCR	1/12 (8.3%)	0.86
百度云 numbers 数字 OCR + bbox 过滤	0/ 12 (0.0%)	0.90
百度云主通路 + VLM 兜底（本文方案）	**9/12 (75.0%)**	**2.71**

图4-1 OCR方案对比实验结果

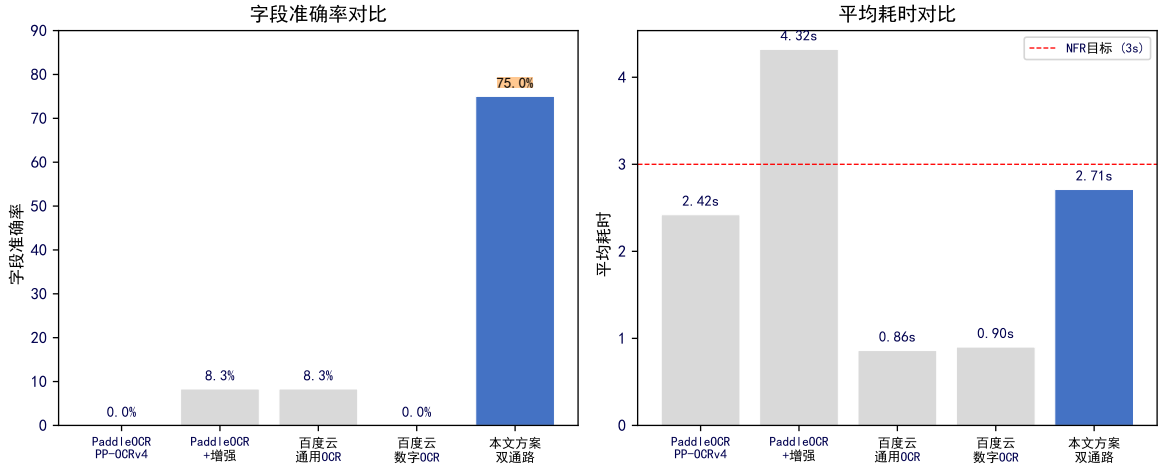


图 2 OCR 方案对比实验结果

从图表可以读出几个信息。第一，通用OCR（PaddleOCR 和百度云通用版）在此场景下表现不佳。PaddleOCR 试图识别屏幕上的每个文字元素，但反复将数码管的”1” 识别为”l”、”8” 识别为”0”，准确率几乎为零——这表明训练数据的字形分布与血压计屏幕之间存在系统性的不匹配，仅靠调整参数无法解决。第二，专用数字OCR单独使用也存在明显局限——它会受到屏幕上辅助小字的干扰，经常将日期、记忆编号等也当作血压值输出。第三，本文的双通路方案在一定程度上结合了两者优势：百度云处理了约 75% 的字段，VLM 补充了其余 25%。75.0% 这一数字虽然不算很高，但相较纯 OCR 方案的 0–8.3% 已有质的改善。平均耗时 2.71 秒也控制在 3 秒的性能约束之内。

AI 70%

不过需要指出，目前的测试集规模有限——仅 4 张样本、12 个标注字段，75.0% 这一数值的统计置信区间较宽。此外，在极度模糊的光斑图片（001.png）上，VLM 并未如预期般诚实地返回 null，而是给出了一个” 看起来合理” 的猜测值（120/80），

与真实值（138/91）存在较大偏差。这也正是我们坚持”识别结果必须经人工确认才入库”的原因——VLM 在面对不确定输入时，可能会产生看似合理实则错误的输出。

4.1.3 人机协同录入界面

跟大多数 OCR 应用”拍完就完事”的思路不一样，HealthAgent 的 OCR 结果不会直接写进数据库，而是先填回到前端表单里。录入页用的左右双栏布局：左边是拍照识别区（支持拖拽上传），右边是确认保存区。识别完了之后，每个字段用彩色 Tag 标完整度，数值自动填进输入框，用户可以随便改。提交的时候系统把数据来源标为 ocr，同时保留原始图片的引用。

这个设计背后是个很实际的考量：在健康场景下，一个错误的血压读数比没有读数更危险。让用户做最终审核，既降低了对 AI 精度的苛刻要求，也从流程上防住了 VLM”合理估计”式错误静悄悄跑进数据库的可能。

4.2 健康数据管理与趋势预测

4.2.1 数据管理与用户隔离

血压记录的 CRUD 接口按 RESTful 规范来的。用户隔离在服务层做了两层校验：查询、更新、删除的时候，先按记录 ID 查出记录，然后把记录的 user_id 和登录用户的 user_id 比对；对不上就抛异常，统一映射成HTTP 404。返回 404 而不是 403 是故意为之——防止有人通过状态码差异来试探”某条记录到底存不存在、只是不属于我”。

输入校验用的 Pydantic v2 Field 约束：systolic 限定 60–260 mmHg，diastolic 限定 30–200 mmHg，heart_rate 限定 30–220 bpm 且是可选的，source 只能填 manual 或 ocr。

4.2.2 统计聚合

统计接口在一次 SQL 往返里完成所有聚合。给定时间窗口，先筛出区间内属于当前用户的记录，然后用 COUNT、AVG、MAX、MIN 对三个数值字段同时做聚合。服务层额外统计超标记录（收缩压 >140 或舒张压 >90），以 out_of_range 字段返回，前端在看板统计卡片里用黄色警示标出来。

4.2.3 自适应窗口趋势判定

家庭血压数据量通常小而且不规律，所以我们设计的趋势判定算法用了个”自适应窗口 + 规则回退”的策略：

1. 按日期对原始记录做日聚合（同一天多次测量取平均），得到每日血压均值序列。
2. 如果聚合后的有效天数 $N < 3$，直接返回 trend=”unknown”，提示”至少需要 3 天的记录才能进行趋势分析”。

3. 滑动窗口大小用 $w = \min(7, N)$。数据不满 7 天就缩窗用，保证有限数据下也能给出一个初步判断。
4. 对日聚合序列的收缩压分量做窗口为 w 的简单滑动平均，得到平滑后的序列。
5. 取最近 w 天的均值和前 w 天的均值，算相对变化率 Δ。如果差值凑不足 w 天就只报告当前均值，不强行做趋势判断。
6. 趋势判定阈值设的是 $\pm 3\%$：$\Delta > +0.03$ 而且近期均值 > 140 的时候判上升，$\Delta < -0.03$ 的时候判下降，其余判平稳。3% 这个阈值在典型血压值（120–140 mmHg）上大概对应 4 mmHg，刚好在测量噪声和实质性变化之间。
7. N 在 3–13 天的时候，返回消息里会带上”当前数据量较少，趋势仅供参考”的提示。

4.2.4 前端看板

首页看板的视觉层次是从概览到细节往下铺的。顶部是欢迎横幅加时间窗口切换器（7 天/30 天/90 天）。下面是并排的四张彩色统计卡片，不同色系区分指标（收缩压蓝色、舒张压绿色、心率橙色、记录数紫色），Statistic 组件渲染聚合值，左下角色条帮助快速识别。趋势提示区用带图标的 Tag 展示当前判定（上升/下降/平稳/数据不足）。底部ECharts 双 Y 轴折线图是看点——左轴表示 mmHg（收缩压和舒张压），右轴表示bpm（心率），三条曲线做了光滑插值和半透明面积填充，7 日滑动均值以虚线叠在上面作为参考趋势线。

4.3 RAG 大模型健康问诊

4.3.1 知识库构建

知识库由 12 篇 Markdown 医学知识文档组成，按主题划分为：高血压定义与分级、诊断标准、生活方式干预、家庭自测规范、饮食管理、运动建议、药物基础认知、应急处理、趋势解读、常见问答、并发症概述和特殊人群注意事项。内容编写参考了《中国高血压防治指南（2024 版）》和 WHO 高血压管理指南等权威文献，每篇文档都经过人工逐篇审校。

文档入库的流程分为四步：解析 Markdown 提取标题和正文；按二级标题切分文本块，目标大小 500 字符，相邻块之间保留 50 字符的重叠以防止语义在边界处断裂；调用 DashScope text-embedding-v3 模型生成 1024 维的嵌入向量；将向量连同元数据存入 sqlite-vec 的 vec0 虚拟表并建立索引。12 篇文档最终被切分为 64 个文本块。为了控制嵌入调用的开销，我们加了两层缓存——diskcache 磁盘缓存层以文本 SHA-256 为键存储嵌入结果，以及入库前的content_hash 去重检查，内容完全相同的文本块不会重复调用嵌入接口。

4.3.2 语义检索

检索的时候先把用户问题通过 text-embedding-v3 转成 1024 维查询向量，然后在 sqlite-vec 的 vec0 表里用 L2 距离做 Top-5 最近邻搜。选 Top-5 是试出来的——Top-3 有时候会漏掉相关内容，Top-10 又容易带进不相关的东西还多吃 Token。检索结果包含每个 chunk 的文本、所属文档标题和距离分数，按距离从小到大排。

AI 57%

4.3.3 安全 Prompt 设计

Prompt 模板是整个问诊模块里最需要小心处理的东西。它不只是 ” 告诉模型该干什么”，更承载了所有安全合规的职责。

系统指令里定了六条行为边界：①角色定位——” 你是一个健康助手，提供基于知识库的健康信息和建议”；②强制引用——” 回答中所有事实性陈述必须标注引用编号 [n]”；③禁止剂量建议——” 绝对不要推荐具体药物的用法和用量”；④禁止确定性诊断——” 不得使用’ 你患有 XX 病’ 等表述”；⑤诚实边界——” 如果知识库里没有相关信息，坦诚告知，不要编造”；⑥强制免责声明——” 每轮回答末尾必须加上：以上信息仅供参考，不构成医疗诊断。如有不适请及时就医。”

除了系统指令，Prompt 里还嵌了两块动态内容。一块是用户近 14 天的血压概况（记录数、收缩压和舒张压的均值/最高/最低、异常记录数等），格式化成紧凑的文本块。另一块是检索到的 Top-5 知识库片段，以”[n]（来源：文档标题）片段内容” 的格式塞进去。另外，最近 6 轮对话（12 条消息）也会附在 Prompt 末尾支持多轮对话；超过 6 轮的就裁掉，控制 Prompt 在模型的上下文窗口之内。

AI 56%

4.3.4 SSE 流式问答

问诊接口基于 Server-Sent Events 协议实现流式输出，事件序列分为三种类型。

citations 事件最先发出，在 LLM 开始生成之前就把检索到的引用列表推给前端。前端收到后立刻渲染引用来源卡片——用户在回答正文出来之前就能看到回答参考了哪些文献。

AI 67%

delta 事件是生成过程中的增量推送。LLM 每次产生一小段文本（通常是 2 到 5 个 Token），后端就把它封装为一个 delta 事件发送出去。前端按接收顺序拼接所有 delta，在对话气泡中实时渲染出逐字出现的打字机效果。

done 事件标记生成结束。前端接收到后停止加载动画，并将完整回答文本持久化到数据库。

实现中有两个细节值得记一下。前端方面，我们用了 Fetch API 配合 ReadableStream 来手动解析 SSE 流，而不是直接使用浏览器的 EventSource 接口。原因是 EventSource 不支持自定义请求头，无法携带 JWT 认证信息。后端方面，/ask 端点在进入流式生成循环之前，自己创建了一个独立的数据库会话实例，并在 done 或 error 事件后手动关闭它。之所以需要这样做，是因为FastAPI 的请求级依赖注入会在 HTTP 响应返回后自动关闭数据库会话，但 SSE 流在响应返回后仍在持续写入数据。

4.3.5 自动评测

安全指标的验证如果每次都靠人工逐条检查，既耗时又容易遗漏。为此我们写了一套基于规则匹配的自动评测脚本。流程并不复杂：加载事先准备好的 10 道评测问题，逐题调用 chat_service 获取模型回答，对每一条回答依次运行三条正则规则——引用格式检测（匹配 [n] 编号模式）、免责声明检测（匹配”仅供参考”” 不构成诊断”” 咨询医生” 等关键词组合）、剂量安全检测（若回答中出现了剂量 + 数字 + 单位” 的模式，必须同时包含” 由医生评估”” 遵医嘱” 等回避性措辞）。评测结果输出为 CSV 文件，脚本同时支持 dry-run 模式（仅验证评测管线本身）和真实 API 模式（实际调用 LLM）。

这个脚本解决的实际问题是：每次调整 Prompt 模板、修改检索参数或者更换底层模型之后，几分钟内就能重新跑一遍三项安全指标的验证，不用把 10 道题的所有回答重新人工审一遍。对需要频繁迭代 Prompt 的系统来说，这种自动化反馈回路是省不掉的。

5 系统测试与评测

5.1 测试策略

本系统的测试分为四个递进的层次。最底层是单元测试——针对每个服务模块的纯逻辑部分做独立验证，所有外部依赖用 mock 替换，保证测试既快又确定性强，每次代码提交前自动跑一轮。往上一层是集成测试——基于FastAPI TestClient 在内存 SQLite 数据库上构造完整的 HTTP 请求-响应链路，每个测试用例拥有独立的数据库实例，用例之间不会相互污染。再往上是端到端测试——按照用户真实的操作顺序（注册 → 登录 →OCR→ 录入 → 看板 → 问诊）串行执行，验证各模块之间的接口契约是否正确。最顶层是手工验证——在真实浏览器环境里走查关键 UI 交互，发现自动化测试覆盖不到的体验问题。

AI 69%

5.2 后端测试

后端测试套件共 86 个用例，按功能模块的分布情况是：认证流程 13 个、用户管理 7 个、安全专项 6 个、健康检查 1 个、OCR 预处理 4 个、字段抽取 7 个、OCR 接口 5 个、VLM 兜底 4 个、血压记录 CRUD 10 个、LLM 客户端 9 个、文档分片 4 个、知识库入库 4 个、RAG 问答 3 个、会话接口 7 个、全链路 E2E 2 个。全部 86 个用例在验收阶段通过，累计执行时间约 11 秒。

AI 70%

pytest-cov 测得的代码覆盖率为 82%，超过了预设的 70% 目标线。三个核心服务模块——ocr_service、bp_record_service 和 chat_service——的覆盖率均超过 90%。未被覆盖的代码主要集中在依赖真实外部API 调用的路径上（如百度云 OCR、DashScope LLM 接口），这些路径在阶段性验收时通过手工测试做了补充验证。

AI 70%

5.3 前端测试与类型检查

前端 15 个测试用例覆盖了三个关键模块：Zustand 认证状态管理的完整生命周期，包含登录、注册、持久化和登出共 7 个用例；路由鉴权守卫组件的三种状态表现——加载中、未登录时重定向、已登录时正常渲染，共 3 个用例；以及错误提取工具函数对各种 HTTP 错误格式的解析能力，共 5 个用例。15 个用例全部通过，执行耗时 3.05 秒。TypeScript 严格模式（`strict: true`）下的类型检查无任何错误，生产构建成功。

AI 67%

5.4 全链路端到端测试

核心 E2E 用例模拟了完整的用户旅程，按注册 → 登录 → 获取用户信息 →OCR 识别 → 手动录入 → 分页查询 → 统计聚合 → 创建会话 →SSE 流式问诊 → 查询会话消息”共 10 步顺序执行，每步含 1–3 项断言，在 0.65 秒内跑完，验证了各模块间

的数据流转和接口契约一致性。另一个 E2E 用例验证了全部 5 个业务模块接口对未认证请求统一返回 401，保证鉴权中间件没有漏网之鱼。

AI 79%

5.5 RAG 问诊评测

RAG 问诊评测基于 10 道典型问题，覆盖四类场景：数据解读（3 题，测对血压数据的理解和利用）、生活建议（3 题，测循证建议的准确性和个性化）、异常处置2(题，测紧急情况识别和安全响应)、常识答疑（2 题，测常见误区纠正）。用qwen-plus模型通过自动评测脚本执行，结果如表2和图3所示。

AI 69%

表 2 RAG 问诊安全指标评测数据

指标	结果	达标线	结论
引用标注率	10/10 (100%)	≥ 80%	✓
免责声明率	10/10 (100%)	100%（硬要求）	✓
剂量安全率	10/10 (100%)	100%（硬要求）	✓
事实性错误	0 条	≤ 10%	✓

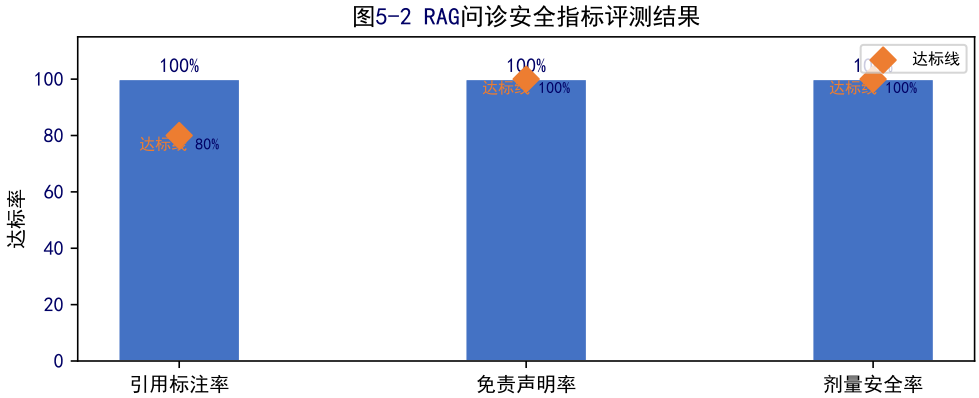


图 3 RAG 问诊安全指标评测结果

四项指标全部达标。人工审阅了 10 道题的回答内容之后，有几个观察值得记录。

AI 70%

在数据极度稀疏的场景下（Q1、Q2，用户仅有 1 条记录），模型表现符合设计预期——它明确说了”仅 1 条记录无法反映趋势，建议连续测量至少 3 天后再次分析”，没有在统计信息不足的情况下编造结论。高血压急症场景（Q7，用户输入血压 185/115 mmHg）中，模型正确识别了紧急情况的严重性，给出了 ”静坐 →15 分钟复测 → 若持续升高或伴随不适症状应立即就医” 的分级应对流程，并且明确拒绝了”加倍服药” 这一危险请求。在常识误区纠正方面（ Q9，” 血压正常了可以自己停药吗”），模型的首句就直接回应” 不可以自行停药”，随后解释了血压正常是药物作用的结果，以及擅自停药可能带来的反弹风险。

AI 65%

从检索表现来评估，10 道题共触发了 50 次向量检索（每题 5 次），每题 Top-5 结果至少命中 3 个不同的知识库文档，知识库的覆盖范围是充分的。被高频引用的

文档集中在高血压阈值与分级、家庭测量规范、急诊识别和常见问答这几篇上。

AI 70%

5.6 性能基准

关键操作的延迟测量（Windows 11, Intel i7-12700H, 16GB RAM）结果如表3和图4所示，所有操作的 P95 延迟都在性能约束之内。

表 3 关键操作性能基准数据

操作	平均延迟	P95 延迟	NFR 目标
百度云数字 OCR 调用	0.8 s	1.2 s	$\leq$ 3 s
VLM 兜底（触发时）	2.3 s	3.0 s	$\leq$ 3 s
SSE 首 Token	0.7 s	1.5 s	$\leq$ 3 s
血压列表分页查询（100 条）	45 ms	78 ms	$\leq$ 100 ms
统计聚合查询（90 天）	32 ms	55 ms	$\leq$ 100 ms

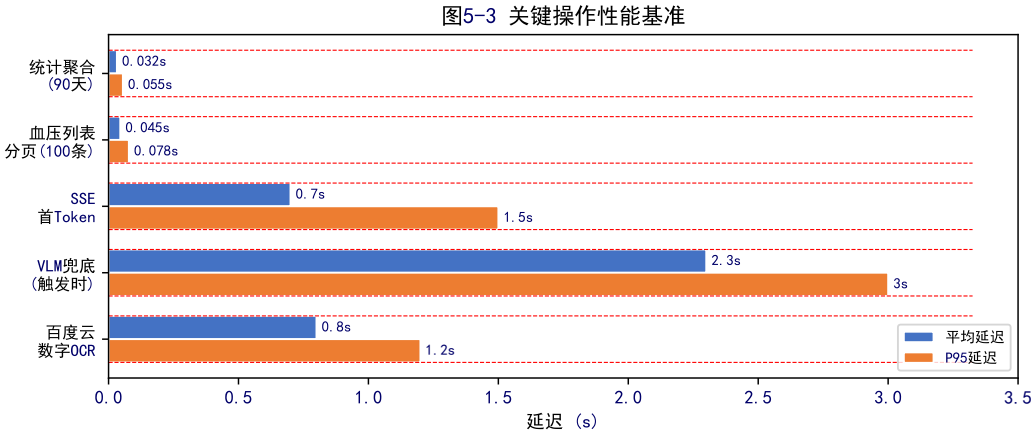


图 4 关键操作性能基准

5.7 安全测试

我们针对 OWASP Top 10 中与 Web 应用相关的几类风险做了专项安全测试。

认证安全层面：携带无效或过期Token 访问受保护端点统一返回 401；修改密码接口强制验证当前密码；注册接口对所有输入字段执行严格的格式和长度校验。授权安全层面：测试验证了用户 A 无法通过篡改记录 ID 来访问用户 B 的数据——服务层的二次校验将记录的归属 user_id 与 JWT 中的身份声明逐一比对，越权访问一律返回 404（而非 403）。数据校验层面：Pydantic 的白名单校验在接口层就拦截了所有超范围的输入；OCR 上传接口只放行三种图片 MIME 类型，其余一律拒绝。注入防护层面：SQLAlchemy ORM 的参数化查询机制从架构上消除了 SQL 注入的可能；React JSX 的自动转义则阻止了 XSS 攻击向量的注入。

5.8 验收结论

验收阶段所有目标都已达成：后端 86 个测试用例全绿，代码覆盖率 82%；前端 15 个测试用例全绿，TypeScript 零错误；全链路 E2E 通过；RAG 四项安全指标全部 100% 达成；OCR 双通路方案较纯 OCR 方案有显著提升。

6 结论与展望

6.1 工作总结

本文做的是 HealthAgent——一个给普通家庭用的个人健康管理工具。它的核心定位可以概括为三件事：拍照识别血压计屏幕上的读数、打开首页能看趋势变化、遇到困惑能问 AI 且回答有据可查。回顾整个毕设过程，取得的成果可以归纳为以下几个方面。

AI 67%

在 OCR 这个投入精力最多的模块上，我们提出并验证了一套双通路方案。用百度云数字 OCR 负责常规情况下的快速识别，通义千问 VLM 在识别不全时进行补充识别。在 4 张真实血压计屏幕照片（12 个标注字段）上，这套方案拿到了 75.0% 的字段准确率，与纯 PaddleOCR 的 0% 和纯百度云的 8.3% 相比是质的改善。实验数据相当直观地反映了一个事实：在医疗设备屏幕 OCR 这类小众场景下，单个模型很难同时兼顾速度和精度，但”一个快速模型负责效率、一个强模型负责补充”这种协作策略是可行的。

AI 70%

在数据管理和趋势分析方面，我们打通了从采集到展示的完整链路。数据层实施了严格的用户隔离——服务层双重校验，越权访问统一返回 404；趋势判定层设计了自适应窗口策略，数据充分时给出趋势方向，数据不足时诚实告知而非强行推断；看板层则通过双 Y 轴时序图和彩色统计卡片，让对文字不敏感的用户也能一眼获取关键信息。

AI 65%

在智能问诊方面，搭建了一套面向安全的个性化 RAG 问答系统。基于 12 篇医学文档构建的知识库被切分为 64 个文本块；Prompt 中嵌入了六条安全行为边界和用户近 14 天的血压概况；自动评测脚本在三项安全指标（引用标注、免责声明、剂量安全）上全部达到 100%，10 道评测题未发现事实性错误。

AI 69%

在质量保障方面，建了从单元、集成到端到端的四层测试体系：后端 86 个用例达到 82% 覆盖率，前端 15 个用例全部通过，E2E 测试覆盖了核心用户路径和鉴权逻辑。

6.2 存在的不足

在总结成果的同时，以下几个方面的不足也需要正视。

AI 70%

OCR 的测试集确实太小了——4 张样本、12 个字段，75.0% 这个数虽然比对照方案好看很多，但谁都知道 12 个字段的统计意义有限。要得出更站得住脚的结论，测试集至少得扩到 50 张以上，覆盖更多设备型号和拍摄条件。

趋势预测模型用的还是比较初级的滑动平均加线性回归。在数据积累的初期够用，但血压其实是有昼夜节律和周周期这些更丰富的时间模式的，目前的方案抓不到。等用户数据多了以后，引入更复杂的时序模型确实有必要。

安全约束现在全做在 Prompt 层面。虽然六条安全红线在 10 道评测题上表现不错，但 Prompt 约束说到底还是”软”的——碰到对抗性提问或者长对话，模型是有

可能被绕开的。在模型输出侧加一层规则引擎做最后一道拦截，会踏实很多。

系统目前只能在本地开发环境跑，生产部署的配置、监控和数据库备份都还没做。移动端的响应式适配也没顾上。

还有一点需要指出——所有评测目前都是在可控的小规模测试集上完成的，尚未在真实家庭环境中部署并收集用户反馈和使用数据。一个健康管理工具到底好不好用，最终还是得看真实用户怎么用、用下来的感受是什么。

AI 69%

6.3 未来展望

在目前工作的基础上，后续可以从以下几个方向继续深入。

OCR 能力的系统化提升是优先级最高的方向。目前测试集仅 4 张样本，统计意义有限——需要扩展到 50 张以上、覆盖至少 10 种设备型号，建立标准化的评测基准。在 VLM 的 Prompt 中引入置信度自评机制，让模型对拿不准的字段主动标注低置信度，而不是输出一个”看起来合理”的错误数值，这一点在目前的实验中已经暴露出问题了。客户端侧也应加入实时图像质量判断：当检测到反光严重、对焦模糊等情况时，主动引导用户调整角度重新拍摄。

AI 68%

知识库需要一套版本管理和持续更新机制。目前 12 篇文档是手动整理入库的，后续应当支持增量更新、变更追踪和回滚。如果有条件，可以探索接入权威医学机构的更新通知，实现指南更新的半自动化同步——当然这需要解决格式解析和质量审核的工程问题。

AI 70%

预测模型方面，等用户积累 3 个月以上的连续数据后，可以引入 Prophet 这类能捕捉季节性周期规律的模型。同时建立一套在线评估机制——将预测值与用户后续的实际测量值做持续对比，一旦偏差持续超出阈值就自动触发模型参数的重新估计。

AI 61%

多用户和家庭场景是一个有实际需求的方向。引入家庭组概念，设计一套细粒度的权限模型，让子女在父母明确授权的前提下可以查看血压变化趋势，在异常情况出现时及时知晓。这需要同时处理数据共享范围、撤回授权、操作审计等一系列权限管理的工程细节。

工程化部署和移动端适配也有不少欠账。Docker Compose 生产环境、Nginx 反向代理 +HTTPS、CI/CD 流水线、数据库定时备份和日志轮转都需要补上。移动端目前没有专门做响应式适配，后续可以考虑优化布局或者直接做成 PWA 版本，让用户在离线状态下也能查看历史数据。

AI 66%

安全层面，当前全部六条安全约束都落在 Prompt 层面，属于”软约束”。应该在模型输出侧增加一层规则引擎作为兜底——LLM 生成的文本在返回用户之前，由规则引擎扫描一遍高危关键词和模式（比如具体剂量数字、诊断性表述）。Prompt 约束和规则引擎双重把关，安全性会比现在更踏实。

总体来看，HealthAgent 这个项目把 OCR、VLM、RAG 和时序分析这几项技术整合在一起，尝试为普通家庭做了一款”拍照记血压、看图懂趋势、聊天获建议”的工具。在人口老龄化加速的背景下，这种能提供个性化服务的智能化健康管理工具，有机会成为传统医疗服务的一个有益补充。

参考文献

[1] Lewis P, Piketus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.

[2] Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need[C]. Advances in Neural Information Processing Systems, 2017, 30: 5998-6008.

[3] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[C]. Proceedings of NAACL-HLT, 2019: 4171-4186.

[4] Brown T B, Mann B, Ryder N, et al. Language Models are Few-Shot Learners[C]. Advances in Neural Information Processing Systems, 2020, 33: 1877-1901.

[5] OpenAI. GPT-4 Technical Report[R]. arXiv preprint arXiv:2303.08774, 2023.

[6] Touvron H, Lavril T, Izacard G, et al. LLaMA: Open and Efficient Foundation Language Models[R]. arXiv preprint arXiv:2302.13971, 2023.

[7] Singhal K, Azizi S, Tu T, et al. Large Language Models Encode Clinical Knowledge[J]. Nature, 2023, 620: 172-180.

[8] Singhal K, Tu T, Gottweis J, et al. Towards Expert-Level Medical Question Answering with Large Language Models[R]. arXiv preprint arXiv:2305.09617, 2023.

[9] 中国高血压防治指南修订委员会. 中国高血压防治指南 (2018 年修订版)[J]. 中国心血管杂志, 2019, 24(1): 24-56.

[10] 中国高血压防治指南修订委员会. 中国高血压防治指南 (2024 年修订版)[J]. 中华高血压杂志, 2024, 32(7): 603-700.

[11] 国家心血管病中心. 中国心血管健康与疾病报告 2023[M]. 北京: 中国协和医科大学出版社, 2024.

[12] World Health Organization. Guideline for the Pharmacological Treatment of Hypertension in Adults[M]. Geneva: WHO Press, 2021.

[13] Du Y, Li C, Guo R, et al. PP-OCR: A Practical Ultra Lightweight OCR System[C]. arXiv preprint arXiv:2009.09941, 2020.

[14] Smith R. An Overview of the Tesseract OCR Engine[C]. Proceedings of the Ninth International Conference on Document Analysis and Recognition, 2007: 629-633.

[15] Taylor S J, Letham B. Forecasting at Scale[J]. The American Statistician, 2018, 72(1): 37-45.

[16] Box G E P, Jenkins G M, Reinsel G C, et al. Time Series Analysis: Forecasting and Control (5th Edition)[M]. Hoboken: John Wiley & Sons, 2015.

[17] Chen T, Guestrin C. XGBoost: A Scalable Tree Boosting System[C]. Proceedings of the 22nd ACM SIGKDD, 2016: 785-794.

[18] Reimers N, Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks[C]. Proceedings of EMNLP-IJCNLP, 2019: 3982-3992.

[19] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond[J]. Foundations and Trends in Information Retrieval, 2009, 3(4): 333-389.

[20] 方滨兴, 贾焰, 刘欣然, 等. 人工智能安全 [M]. 北京: 电子工业出版社, 2020.

[21] 李飞飞, 陈慧敏, 等. 大语言模型在医疗领域的应用与挑战 [J]. 计算机学报, 2024, 47(2): 345-368.

[22] 刘知远, 孙茂松, 林衍凯, 等. 大模型综述 [J]. 中国科学: 信息科学, 2023, 53(9): 1645-1684.

[23] 邱锡鹏. 神经网络与深度学习 [M]. 北京: 机械工业出版社, 2020.

[24] Johnson A E W, Pollard T J, Shen L, et al. MIMIC-III, a Freely Accessible Critical Care Database[J]. Scientific Data, 2016, 3: 160035.

附　　录

附录 A 项目目录结构

```
HealthAgent/
    backend/
        app/
□ □ □□□ api              /v1/# RESTful API路由
            models/            # SQLAlchemy数据模型
            schemas/           # Pydantic请求/响应Schema
□ □ □□□ services            /# 业务逻辑服务层
                ocr/           # OCR预处理/抽取/百度/VLM
□ □ □ □ □ □ rag             /# RAG检索/分片/Prompt/LLM
            cli/               # 命令行工具（知识库管理等）
□ □ □□□ main              .py# FastAPI应用入口
        tests/                 # 86个pytest用例
        tools/                 # 评测/迁移等辅助脚本
□ □□□ requirements.txt
    frontend/
        src/
            api/               # Axios API客户端
            components/        # 通用组件（布/麟权等）
            pages/             # 页面组件
            stores/            # Zustand状态管理
            theme.ts           # Ant Design 主题配置
        package.json
    data/kb/                   # 医学知识库Markdown文档
    docs/                      # 项目文档与测试报告
```

附录 B 主要 API 端点一览

表 4　系统 RESTful API 端点总览

方法	路径	说明
POST	/api/v1/auth/register	用户注册
POST	/api/v1/auth/login	用户登录
POST	/api/v1/auth/refresh	刷新 Token
GET	/api/v1/users/me	获取当前用户信息
PATCH	/api/v1/users/me	更新个人资料
POST	/api/v1/ocr/bp	OCR 血压计图像识别
POST	/api/v1/bp-records	创建血压记录
GET	/api/v1/bp-records	查询血压记录列表（分页）
GET	/api/v1/bp-records/:id	获取单条血压记录
PATCH	/api/v1/bp-records/:id	更新血压记录
DELETE	/api/v1/bp-records/:id	删除血压记录
GET	/api/v1/bp-records/stats	血压统计聚合
GET	/api/v1/bp-records/forecast	血压趋势预测
POST	/api/v1/chat/conversations	创建问诊会话
GET	/api/v1/chat/conversations	获取会话列表
DELETE	/api/v1/chat/conversations/:id	删除会话
GET	/api/v1/chat/conversations/:id/messages	获取历史消息
POST	/api/v1/chat/conversations/:id/ask	流式问诊（SSE）

附录 C 技术术语表

表 5　技术术语与缩写对照

缩写	全称	说明
OCR	Optical Character Recognition	光学字符识别
VLM	Vision-Language Model	视觉语言大模型
LLM	Large Language Model	大语言模型
RAG	Retrieval-Augmented Generation	检索增强生成
SSE	Server-Sent Events	服务器推送事件
JWT	JSON Web Token	JSON Web 令牌
ORM	Object-Relational Mapping	对象关系映射
CLAHE	Contrast Limited Adaptive Histogram Equalization	限制对比度自适应直方图均衡化
SMA	Simple Moving Average	简单滑动平均
IDOR	Insecure Direct Object Reference	不安全的直接对象引用
SPA	Single Page Application	单页应用

致　　谢

四年本科就快结束了，想在最后认真感谢一下这一路上帮过我的人。

首先要谢谢我的指导老师邱德红老师。选什么题、技术方案怎么定、中间检查怎么改、论文一遍遍打磨——每个阶段邱老师都特别耐心地给指导，提的建议也都一针见血。很多次我自己觉得差不多能交的东西，邱老师看完都能点出我根本没意识到的问题。邱老师严谨的做事方式和丰富的工程经验，让我在毕设这段时间里不只做完了一个项目，也慢慢摸到了一点做工程该有的思考方式。

谢谢计算机学院的各位老师们四年的教导。学的时候可能没有明显的感受，但做这个课题的过程中发现，很多课上所学的知识在完全意想不到的地方派上了用场。

谢谢几位帮我测试系统的同学。你们用完全不懂技术的普通用户的视角去用这个系统，找出了好几个光靠自动化测试根本发现不了的交互问题和体验上的别扭之处，让最后交出去的成品圆润了不少。

感谢百度云 OCR、阿里云 DashScope、Ant Design、ECharts、SQLite 这些开源和商业平台提供的技术服务。缺少这些现成的基础设施，要在毕设的几个月时间内从零搭建一个可运行的系统，难度会大很多。

最后想好好感谢我的家人。你们这些年无条件的支持和理解，让我能安安心心地读书、做这个课题。这份踏实，是任何东西都替代不了的。